Trường Đại Học Công Nghệ Sài Gòn
Khoa Công Nghệ Thông Tin
---oOo---



Giáo trình thực hành:

KỸ THUẬT LẬP TRÌNH

(Lưu hành nội bộ)

Năm 2024

Bài 1: ÔN TẬP CẤU TRÚC LỆNH RẼ NHÁNH, LỆNH LẶP, HÀM

I. MỤC TIÊU:

Sau khi thực hành xong bài này, sinh viên được ôn tập :

- Ôn tập cấu trúc lệnh rẽ nhánh.
- Ôn tập cấu trúc lệnh lặp.
- Ôn tập hàm.

II. TÓM TẮT LÝ THUYẾT:

1. KHAI BÁO VÀ ĐỊNH NGHĨA HÀM:

- Khai báo nguyên mẫu hàm trong file **thuvien.h**

<kiểu dữ liệu> <tên hàm>([danh sách tham số hình thức] **);**

Ví dụ:

```
#include <iostream>
using namespace std;
float tinhDiemTrungBinh(float toan, float ly, float hoa);
```

- Định nghĩa hàm trong file **caidat.cpp**

<kiểu dữ liệu> <tên hàm>([danh sách tham số hình thức] **)**
{
//Các lệnh trong thân hàm
[**return** gia_tri;]
}

Ví dụ:

```
#include "thuvien.h"
float tinhDiemTrungBinh(float toan, float ly, float hoa)
{
    return (toan * 3 + ly * 2 + hoa) / 6;
}
```

- Gọi hàm trong file **chuongtrinh.cpp** :

<tên hàm>([danh sách tham số thực]);

Ví dụ:

```
#include "thuvien.h"
void main() {
    float toan, ly, hoa, dtb;
    cout << "\n Nhap diem toan, ly, hoa";
    cin >> toan >> ly >> hoa;
    //goi ham tinh diem trung binh
    dtb = tinhDiemTrungBinh(toan, ly, hoa);
    cout << "\n Diem Trung Binh = " << dtb;
    system("pause");
}
```

2. MỘT SỐ QUY TẮC VỀ HÀM

- **Hàm có giá trị trả về: <kiểu dữ liệu>** được khai báo là tên kiểu dữ liệu của giá trị trả về, giá trị này được tính toán trong hàm và được trả về bởi lệnh **return**.
- **Hàm không có giá trị trả về: <kiểu dữ liệu>** được khai báo là **void.**
- **Tham số hình thức và tham số thực:**
  - **Tham số hình thức**: Tham số dùng khi khai báo, và tham số này sẽ được dùng tạm trong quá trình định nghĩa hàm
  - **Tham số thực**: Là tham số được cung cấp khi hàm được sử dụng, tham số này sẽ thay thế vào vị trí của tham số hình thức hay nói cách khác là gán tham số thực tế cho tham số hình thức. Tham số thực còn được gọi là đối số.
  - Ví dụ: Tham số hình thức và tham số thực.

```
//khai báo nguyên mẫu
void ham(int n); //Tham số hình thức
void main()
{
        int a;
        //gọi hàm
        ham(a); // Tham số thực hay đối số
}
//định nghĩa hàm
void ham(int n) { //Tham số hình thức
……..
}
```

- **Cách truyền tham số:**
  - **Truyền theo tham trị**: giá trị của tham số không thay đổi sau lời gọi hàm.
  - **Truyền theo tham biến**: giá trị của tham số bị thay đổi sau lời gọi hàm (nếu trong hàm có lệnh làm thay đổi giá trị tham số).

3. CẤU TRÚC RẼ NHÁNH

CÚ PHÁP	VÍ DỤ
▪ **Cấu trúc if:** **if** (biểu thức điều kiện) { Khối lệnh 1;} **else** { Khối lệnh 2;}	int a = 5, b = 7; **if (a==b)** cout<<"a bang b"; **else** cout<<"a khac b";
▪ **Cấu trúc switch:** **switch** (biểu thức) **{** **case** <giá trị 1>: [lệnh 1;break;] **case** <giá trị 2>: [lệnh 2; break;] ... **case** <giá trị n-1>: [lệnh n-1; break;] [**default**: lệnh n;] **}**	int thang = 4; **switch** (thang) **{** **case** 1: **case** 2: **case** 3: cout<<"Qui I"; break; **case** 4: **case** 5: **case** 6: cout<<"Qui II"; break; **case** 7: **case** 8: **case** 9: cout<<"Qui III"; break; **case** 10: **case** 11: **case** 12: cout<<"Qui IV"; break; **default**: cout<<"Thang sai!"; **}**

4. CẤU TRÚC LẶP

CÚ PHÁP	VÍ DỤ

▪ **Cấu trúc for:**
for (biểu thức 1; biểu thức 2; biểu thức 3)
{ Khối lệnh;}
	int kq = 1;
	for (int i=1; i<=n; i++)
	kq = kq * i;

▪ **Cấu trúc while:**
while (biểu thức điều kiện)
{ Khối lệnh;}
	int kq = 1, i = 1;
	while(i<=n)
	{
	kq = kq * i;
	i++;
	}

CÚ PHÁP	VÍ DỤ

▪ **Cấu trúc**
do …while:
do
{
Khối lệnh;
} while (biểu thức điều kiện);
	int kq = 1, **i** = 1;
	do
	{
	kq = kq * **i**;
	i++;
	} while(i<=n);

III. NỘI DUNG THỰC HÀNH:

1/ Chương trình mẫu: (3 điểm)

➢ **Yêu cầu :** SV hãy viết các hàm :

- Viết hàm giải phương trình bậc 1: ax + b = 0
- Viết hàm tính 2^n . Hướng dẫn :
  VD: n= 4 thì hàm tính 4 lần nhân 2 = 4*4*4*4. Vậy ta sẽ thực hiện vòng lặp với số lần là n và mỗi lần lặp sẽ nhân cho 2.

➢ Cho đoạn mã trong file **lab1_thuvien.h** như sau:

Dòng	Mã lệnh	Ghi chú
1	#include <iostream>	
2	using namespace std;	
3	void giaiPtb1(int a, int b, double& x);	
4	double tinh2MuN(int n);	

➢ Cho đoạn mã trong file **lab1_caidat.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	#include "lab1_thuvien.h"	
2	//Hàm giải phương trình bậc 1: ax + b = 0	
3	void giaiPtb1(int a, int b, double &x)	
4	{	
5	x = -b / a;	
6	}	
7	//Hàm tính 2n	
8	double tinh2MuN(int n)	
9	{	
10	double s = 1;	
11	for (int **i** = 1; **i** < n; i++)	
12	s *= 2;	
13	return s;	
14	}	

➢ Cho đoạn mã trong file **lab1_chuongtrinh.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	#include "lab1_thuvien.h"	
3	void main()	
4	{	
5	int a, b;	
6	double x;	
7	cout<<"\nGiai phuong trinh bac 1(ax+b= 0):";	
8	cout<<"\nNhap a=";	
9	cin >> a;	
10	cout << "\nNhap	
11	b=";	
12	cin >> b;	
13	**giaiPtb1(a, b);**	
14	cout <<"\n\nNghiem cua phuong trinh la:"<<x;	
15	int n;	
16	cout << "\nTinh 2 mu n:";	
17	cout << "\nNhap n=";	
18	cin >> n;	
19	cout << "\n2 mu "<< n <<" = "<< **tinh2MuN(n);**	
20	system("pause");	
	}	

Yêu cầu sinh viên:

1. Hàm **tinh2MuN**(int n) đã sai. SV hãy sửa lại để ra đúng kết quả 2 mũ n.
2. Sinh viên thực thi chương trình với các giá trị nhập vào như sau:
   - a = 1, b = 2, n = 2 : cho biết kết quả? Giải thích tại sao kết quả không ra số thực? SV sửa lại mã lệnh để cho kết quả nghiệm là số thực .
   - a = 0, b = 2, n = 3 : cho biết kết quả? Giải thích tại sao chương trình báo lỗi khi thực thi? SV sửa lại mã lệnh để cho kết quả đúng như sau: nếu a = 0 thì xuất ra màn hình "Không chia cho số 0", ngược lại thì tính nghiệm vào tham số x.
3. Viết lại hàm **tinh2MuN**(int n) bằng vòng lặp while.

2/ Bài tập vận dụng: (2 điểm)

1. Hãy sửa lại hàm **tinh2MuN** để tính được cả số mũ âm, lưu ý: $\mathbf{2^{-n} = 1 / (2^n)}$
2. Hãy sửa lại hàm **void** giaiPtb1(int a, int b, double &x) thành **bool** giaiPtb1(int a, int b, double &x) để cho biết phương trình có nghiệm hay vô nghiệm.

3/ Bài tập tổng hợp: (3 điểm)

1. Viết hàm giải phương trình bậc 2 ($ax^2 + bx + c = 0$) :

   int giaiPtb2(int a, int b, int c, double &x1, double &x2)

   - Kết quả trả về :
     - 0 : phương trình vô nghiệm.
     - 1: phương trình có 1 nghiệm.
     - 2: phương trình có 2 nghiệm.
     - 3: phương trình vô số nghiệm
   - x1, x2 : là 2 nghiệm của phương trình.
2. Viết hàm tính x^y , x và y là 2 tham số của hàm.

4/ Bài tập làm thêm: (2 điểm)

1. Cho số nguyên dương n , viết hàm kiểm tra n có phải là lũy thừa cơ số 2 hay không?
   Ví dụ: $n=16 = 2^4$ => 16 là lũy thừa cơ số 2.
2. Viết hàm tìm ước số chung lớn nhất của 2 số nguyên.
3. Viết hàm tìm bội số chung nhỏ nhất của 2 số nguyên.
4. Viết hàm kiểm tra số n có phải là số nguyên tố hay không?
5. Viết hàm xuất ra n số nguyên tố đầu tiên.
   Ví dụ: n = 5 => xuất ra: 2 3 5 7 11
6. Viết hàm xuất ra các số nguyên tố <= n.
   Ví dụ: n = 20 => xuất ra: 2 3 5 7 11 13 17 19
7. Viết hàm kiểm tra số nguyên dương n có phải là số hoàn thiện không?
8. Viết hàm tính tổng các ký số của 1 số nguyên.
   Ví dụ: số 123 có tổng các ký số = 1+2+3 = 6.

-HẾT LAB 1-

Bài 2: KIỂU CẤU TRÚC, MẢNG 1 CHIỀU, GHI ĐỌC FILE

I. MỤC TIÊU:

Sau khi thực hành xong bài này, sinh viên được ôn tập:

- Ôn tập kiểu dữ liệu struct, kiểu mảng một chiều. Xây dựng kiểu dữ liệu chứa mảng 1 chiều.
- Nhập/xuất mảng 1 chiều số nguyên trên thiết bị nhập/xuất chuẩn.
- Ghi/đọc file mảng 1 chiều với file văn bản.
- Phép duyệt dãy số và một số phép toán cơ bản trên dãy số.

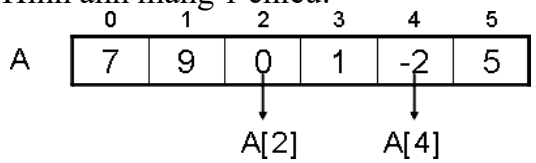
II. TÓM TẮT LÝ THUYẾT:

1. KIỂU CẤU TRÚC

CÚ PHÁP	VÍ DỤ
▪ Khai báo: **struct <tên kiểu> {** **thành phần 1;** **thành phần 2;** **…** **thành phần n;** **};**	Khai báo kiểu SoPhuc gồm 2 thành phần: thực và ảo **struct SoPhuc {** **float thuc;** **float ao;** **};**
▪ Khai báo biến kiểu cấu trúc: **<kiểu cấu trúc> <danh sách các biến>;**	**SoPhuc a, b;**
▪ Truy cập đến các thành phần kiểu cấu trúc: **<biếu kiểu cấu trúc>.<tên thành phần>**	**b.ao, b.thuc, a.ao, a.thuc**

2. MẢNG MỘT CHIỀU:

- Hình ảnh mảng 1 chiều:



CÚ PHÁP	VÍ DỤ
Khai báo: <kiểu dữ liệu> <Tên mảng>[Số phần tử tối đa];	int a[50];
Hoặc: <kiểu dữ liệu> <Tên mảng>[ ] = {ds các giá trị};	int b[ ] = {1,2,3,4,5};

Truy cập đến các phần tử của mảng: <Tên mảng>[chỉ số] Trong đó: 0<= chỉ số < số phần tử tối đa	a[2] => truy xuất đến phần tử thứ 3 của mảng, có giá trị là 0 a[4] => truy xuất đến phần tử thứ 5 của mảng, có giá trị là -2

3. XÂY DỰNG KIỂU DỮ LIỆU DÃY SỐ

a. Ý tưởng:

Ta thấy đối với mảng một chiều, muốn duyệt mảng phải biết được số phần tử, vì vậy, để cho đơn giản, ta tạo ra kiểu dữ liệu DaySo có 2 thành phần là mảng 1 chiều và số phần tử của nó.

b. Cài đặt:

```
const int KTM = 50; // Khai báo số phần tử tối đa của DaySo
struct   DaySo
{
   int list[KTM];
   int n; //số phần tử mảng
};
```

c. Duyệt qua các phần tử của dãy số

Cho dãy số a có n phần tử, phần tử đầu tiên có chỉ số là 0, phần tử sau cùng có chỉ số là n-1, ta có phép duyệt các phần tử của dãy số tổng quát như sau:

```
for (int i = 0; i <a.n; i++)     // hoặc i <= a.n - 1
{
        // Xử lý trên phần tử a.list[i]
}
```

4. ĐỌC FILE, GHI FILE VĂN BẢN

Mở file:

- Khai báo con trỏ file:

  FILE* fp; *//con trỏ fp kiểu FILE*

- Mở file

  fp= fopen(path, mode);

  - path: đường dẫn đến file muốn mở
  - mode: chế độ mở gồm các chế độ sau:
    - **"r":** mở chế độ đọc, file phải tồn tại
    - **"w":** tạo file mới để ghi. Nếu file có sẵn thì bị ghi đè.
    - **"a":** mở để ghi tiếp nội dung, nếu file không có sẵn thì tạo file mới.

  Hàm trả về con trỏ file nếu mở được file, trả về **NULL** nếu không mở được file.

Đọc file

fscanf(fp, "chuỗi đặc tả",[&biến 1, &biến 2…];

- fp: là con trỏ file
- "chuỗi đặc tả": **%d**: số nguyên, **%f**: số thực, **%ld**: là số nguyên long, **%c**: ký tự biến 1, biến 2: là các biến muốn lưu giá trị từ file

Ghi file

fprint(fp, "chuỗi đặc tả",[biểu thức 1,biểu thức 2,…];

- fp: là con trỏ file
- "Chuỗi đặc tả": **%d:** số nguyên int, **%f**: số thực, **%ld**: số nguyên long, **%c**: ký tự biểu thức 1, biểu thức 2: là các giá trị muốn ghi ra file.

Lưu ý: Phải sử dụng #define _CRT_SECURE_NO_WARNINGS vì một số hàm xử lý file được cảnh báo là kém an toàn vì nó có thể gây tràn bộ nhớ đệm.

III. NỘI DUNG THỰC HÀNH:

1. **Chương trình mẫu:** Chương trình nhập/xuất, đọc/lưu dãy số

➢ Cho đoạn mã trong file **lab2_thuvien.h** như sau:

Dòng	Mã lệnh	Ghi chú
1	`#include <iostream>`	
2	`using namespace std;`	
3	`const int KTM=50;`	
4	`struct DaySo{`	
5	`  int arr[KTM];`	
6	`  int n;`	
7	`};`	
8	`bool docDaySo(DaySo &ds, char *tenFile);`	
9	`bool luuDaySo(DaySo ds, char *tenFile);`	
10	`void nhapDaySo(DaySo &ds);`	
11	`void xuatDaySo(DaySo ds);`	
12	`int tongDS(DaySo ds);`	

➢ Cho đoạn mã trong file **lab2_caidat.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	`#include "lab2_thuvien.h"`	
2	`void nhapDaySo(DaySo &ds)`	
3	`{`	
4	`  do{`	
5	`  cout<<"Nhap so phan tu: ";`	
6	`  cin>>ds.n;`	
7	`  }while(ds.n<=0);`	
8	`  for (int i=0; i<ds.n; i++) {`	
9	`  cout<<"Nhap phan tu thu "<<i<<" : ";`	
10	`  cin>>ds.arr[i];`	
11	`  }`	
12	`}`	
13	`void xuatDaySo(DaySo ds)`	
14	`{`	
15	`  for (int i=0; i<ds.n; i++)`	
16	`  cout<<ds.arr[i]<<" ";`	
17	`  cout<<endl;`	
18	`}`	
19	`bool docDaySo(DaySo &ds, char *tenFile)`	

```
20  {
21      FILE *f;
22      f=fopen(tenFile, "r");
23      if(f==NULL)
24          return false;
25      fscanf(f, "%d",&ds.n);
26      for (int i=0; i<ds.n; i++)
27          fscanf(f, "%d",&ds.arr[i]);
28      fclose(f);
29      return true;
30  }
31  bool luuDaySo(DaySo ds, char *tenFile)
32  {
33      FILE *f;
34      f=fopen(tenFile, "w");
35          if(f==NULL)
36      return false;
37      fprintf(f, "%d\n",ds.n);
38      for (int i=0; i<ds.n; i++)
39          fprintf(f, "%d\t",ds.arr[i]);
40      fclose(f);
41      return true;
42  }
43  int tongDS(DaySo ds)
44  {
45      int tong=0;
46      for (int i=0; i<ds.n; i++)
47          tong+=ds.arr[i];
48      return tong;
49  }
```

//Khi DaySo luu xuong file text sẽ có dạng

5 => số lượngphần tử
9 2 6 3 8 => giá trị các phần tử

//khi đọc file cũng sẽ theo thứ tự:
1. Đọc số lượng
2. Đọc từng giá trị

➢ Cho đoạn mã trong file **lab2_chuongtrinh.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	#include "lab2_thuvien.h"	
2	int main()	
3	{	
4	DaySo x;	
5	nhapDaySo(x);	
6	cout<<"Day so vua nhap la: "<<endl;	
7	xuatDaySo(x);	

```
8       cout<<"Tong cac phan tu cua day so la: ";
9          cout<<tongDS(x)<<endl;
10         //Luu day so thanh file
11         if(luuDaySo(x, "dayso.txt"))
12            cout<<"Day so da duoc luu"<<endl;
13         else
14            cout<<"Luu khong thanh cong"<<endl;
15         return 0;
16      }
```

Yêu cầu sinh viên: (3 điểm)

1. SV hãy chạy chương trình và lưu lại dãy số vừa nhập.
2. SV hãy thay đổi hàm main để đọc dãy số vừa lưu.
3. Từ hàm **tongDS**, SV hãy viết hàm **tichDS** để tính tích các phần tử trong dãy số.
4. Từ hàm **tongDS**, SV hãy viết hàm **demPTDuong** để đếm các phần tử dương trong dãy số.
5. Viết hàm xuất ra các phần tử có giá trị là số chẵn trong DaySo

2. Bài tập vận dụng: (2 điểm)

1. SV hãy sửa lại hàm **tongDS** để tính tổng các phần tử chẵn trong dãy số.
2. SV hãy viết hàm **tichPTChiSoChan** để tính tích các phần tử có chỉ số chẵn trong dãy số.
3. SV hãy viết hàm **demPTChiaHet3Va5** để đếm các phần tử chia hết cho 3 và 5 trong dãy số.

3. Bài tập tổng hợp: (3 điểm)

1. Viết hàm tính tổng các phần tử là số nguyên tố có trong dãy số.
2. Viết hàm xuất ra màn hình số lượng các phần tử âm, số lượng các phần tử dương, số lượng các phần tử bằng 0 trong dãy số.
3. Viết hàm đếm số lượng số hoàn thiện trong dãy số.
4. Viết hàm đếm số phần tử (không tính trùng nhau) trong dãy số.
5. Viết hàm xuất ra các phần tử khác nhau và số lượng phần tử đó có trong dãy số

4. Bài tập làm thêm: (2 điểm)

1. Xây dựng cấu trúc dãy số thực
2. Viết các hàm nhập, xuất, đọc, lưu cho dãy số thực
3. Viết hàm chỉ xuất ra các số chính phương có trong dãy số thực
4. Viết hàm lưu 1 dãy số thực thành 2 file:
   - Thuc_am.txt: chỉ chứa các số âm

- Thuc_duong.txt chỉ chứa các số dương

5. Viết hàm tách các số nguyên có trong dãy số thực ra thành dãy số nguyên của phần trên.

-HẾT LAB 2-

Bài 3: MẢNG 1 CHIỀU – CÁC PHÉP TOÁN TRÊN MẢNG 1 CHIỀU

I. MỤC TIÊU:

Sau khi thực hành xong bài này, sinh viên hiểu và vận dụng :

- Thuật toán: tìm kiếm
- Thuật toán tìm giá trị lớn nhất và nhỏ nhất.

II. TÓM TẮT LÝ THUYẾT:

1. THUẬT TOÁN TÍNH TỔNG, TÍCH VÀ ĐẾM

Ý tưởng chính để tìm phần tử thuộc tập **S** có giá trị là **K** được thể hiện như sau :

```
Duyệt mọi X thuộc S
    Nếu X = K thì
        Trả về vị trí hiện tại của X
Cuối duyệt
Trả về -1 (mang ý nghĩa là không có phần tử nào bằng với K)
```

2. THUẬT TOÁN TÌM GIÁ TRỊ LỚN NHẤT VÀ NHỎ NHẤT:

Bài toán đặt ra là tìm phần tử nhỏ (hay lớn nhất) trong số các phần tử của tập S thỏa mãn một điều kiện K nào đó được mô tả như sau:

Thuật toán A: nếu có thể chọn x0

```
Chọn x0 thuộc S sao cho thỏa điều kiện K và gán Min = x0
Duyệt x thuộc S (không chứa x0)
    Nếu x thõa K và Min > x thì Min = X
Cuối duyệt
```

Thuật toán B: nếu "khó" chọn x0

```
Flag = false;
Duyệt x thuộc S
    Nếu x thỏa K
        Nếu not Flag thì
            Min = x; Flag = true
        Ngược lại nếu Min > x thì
            Min = x;
        Cuối nếu
    Cuối nếu
Cuối Duyệt
```

Nếu điều kiện K là hằng đúng (true) thì ta không cần kiểm tra, hơn nữa chỉ cần khởi gán biến Min bằng bất kỳ phần tử nào của S (thường là phần tử đầu tiên).

Trong nhiều trường hợp, ta không thể chọn được x0 bởi vì ràng buộc của điều kiện K. Chẳng hạn tìm số dương nhỏ nhất trong mảng a[0], a[1]…a[n-1], khởi đầu ta không thể gán Min bằng a[0] hay số khác (bởi vì chưa biết số nào dương). Thuật toán B thường được áp dụng cho những trường hợp như vậy: chỉ khi nào cờ Flag được chuyển thành true thì Min mới bắt đầu xác định (lúc đó x là x0 ở thuật toán A, nếu kết thúc thuật toán B mà Flag vẫn là false thì S không có phần tử nào thỏa điều kiện K.

III. NỘI DUNG THỰC HÀNH:

1/ Chương trình mẫu: (2 điểm) SV hãy bổ sung bài LAB 2 các hàm:

- Tìm giá trị x trong 1 dãy số
- Tìm giá trị nhỏ nhất trong dãy số
- Tìm giá trị **dương** nhỏ nhất trong dãy số

➢ Tạo file **lab3_thuvien.h**: dựa vào code bài Lab2_thuvien.h (***Lưu ý: chỉ lấy các thông tin cần sử dụng***) và bổ sung thêm nội dung như sau:

Dòng	Mã lệnh	Ghi chú
1	…	
2	//Nội dung file lab2_thuvien	
3	…	
4		
5	int tim(DaySo ds, int x);	
6	int min(DaySo ds);	
7	bool minDuong(DaySo ds, int &min);	

➢ Tạo file **lab3_caidat.cpp**: dựa vào code bài Lab2_caidat.cpp (***Lưu ý: chỉ lấy các hàm cần sử dụng***) và bổ sung thêm các định nghĩa hàm như sau:

Dòng	Mã lệnh	Ghi chú
1	#include " lab3_thuvien.h"	
2	…	
3	//Nội dung các định nghĩa hàm lab2_caidat	
4	…	
5	//lab3	
6	int tim(DaySo ds, int x)	
7	{	
8	for (int i=0; i<ds.n; i++)	

```
9           if(ds.arr[i]==x)
10              return i;
11          return -1;
12      }
13      int min(DaySo ds)
14      {
15          int x=ds.arr[0];
16          for (int i=1; i<ds.n; i++) {
17              if(x>ds.arr[i])
18                  x=ds.arr[i];
19          }
20          return x;
21      }
22      bool minDuong(DaySo ds, int &min)
23      {
24          bool flag=false;
25          for (int i=0; i<ds.n; i++) {
26              if(ds.arr[i]>0)
27                  if(!flag)
28                  {
29                      min=ds.arr[i];
30                      flag=true;
31                  }else if(min>ds.arr[i])
32                      min=ds.arr[i];
33          }
34          return flag;
35      }
```

Yêu cầu: SV hãy bổ sung hàm main để gọi các hàm trên

2/ Bài tập vận dụng: (3 điểm)

1. SV hãy chỉnh sửa lại hàm tìm để tìm vị x cuối cùng có trong dãy số
2. SV hãy sửa lại hàm min thành **max** để tìm giá trị lớn nhất trong dãy số.
3. SV hãy sửa lại hàm **minDuong** thành hàm **maxAm**để tìm giá trị âm lớn nhất trong dãy số.

3/ Bài tập tổng hợp: (3 điểm)

1. Viết hàm tìm vị trí phần tử có giá trị lớn nhất trong dãy số
2. Viết hàm đếm các phần tử có giá trị lớn nhất trong dãy số
3. Viết hàm tìm vị trí phần tử có giá trị nhỏ nhất trong dãy số
4. Viết hàm tính tổng các phần tử là số nguyên tố trong dãy số.

5. Viết hàm tìm số nguyên tố nhỏ nhất trong dãy số.

4/ Bài tập làm thêm (2 điểm)

6. Viết hàm cho phép đếm số phần tử (không tính trùng nhau) trong dãy số.
7. Viết hàm đếm số lượng số hoàn thiện trong dãy số.
8. Viết hàm tìm số hoàn thiện lớn nhất trong dãy số.

-HẾT LAB 3-

Bài 4: CÁC PHÉP TOÁN TRÊN MẢNG 1 CHIỀU (tiếp theo)

I. MỤC TIÊU:

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Xử lý file nhị phân
- Các phép toán trên danh sách : chèn, loại bỏ, sắp xếp, trộn, tách, ghép,...

II. TÓM TẮT LÝ THUYẾT:

1. ĐỌC FILE, GHI FILE NHỊ PHÂN

Mở file:

– Khai báo con trỏ file:

FILE* fp; *//con trỏ fp kiểu FILE*

– Mở file

fp= fopen(path, mode);

- path: đường dẫn đến file muốn mở
- mode: chế độ mở gồm các chế độ sau:
  - **"rb":** mở chế độ đọc, file phải tồn tại
  - **"wb":** tạo file mới để ghi. Nếu file có sẵn thì bị ghi đè.
  - **"ab":** mở để ghi tiếp nội dung, nếu file không có sẵn thì tạo file mới.

Hàm trả về con trỏ file nếu mở được file, trả về **NULL** nếu không mở được file.

Đọc file

*int fread(void *pt, int size, int n, File *fp);*

- ptr: con trỏ trỏ đến vùng nhớ chứa dữ liệu đọc được.
- size: kích thước của kiểu dữ liệu (byte).
- n: số phần tử đọc từ file.
- fp: con trỏ kiểu file.

Ghi file

*int fwrite(void *pt, int size, int n, File *fp);*

- pt là con trỏ trỏ tới vùng nhớ chứa dữ liệu cần ghi
- size là kích thước của mẫu tin (byte)

- n làsố mẫu tin
- fp là con trỏ File.

2. CÁC PHÉP TOÁN TRÊN DANH SÁCH

▪ **Chèn một phần tử vào danh sách**

- Hàm chenXVaoViTri:
  - Đầu vào :
    - DaySo : dãy số cần chèn
    - viTri : vị trí cần chèn
    - x : số cần chèn
  - Đầu ra : hàm trả về 1 nếu chèn thành công, ngược lại trả về 0.
- Thuật giải :

```
Duyệt qua danh sách arr từ phần tử cuối đến phần [viTri]
    Gán a.arr[i] = a.arr[i-1]
Gán a.arr[i] = x
Số phần tử của dãy tăng 1
```

▪ **Xóa phần tử**

- Hàm xoaPhanTu:
  - Đầu vào :
    - DaySo : dãy số cần xóa
    - viTri : vị trí cần xóa
  - Đầu ra : hàm trả về 1 nếu xóa thành công, ngược lại trả về 0.
- Thuật giải :

```
Nếu viTri<0 hoặc viTri>=n thì trả về 0
Duyệt qua dãy số a từ phần tử [viTri] đến phần tử kế cuối
    Gán a.arr[i] = a.arr[i+1]
Số phần tử của dãy giảm 1
```

Bài 4: CÁC PHÉP TOÁN TRÊN MẢNG 1 CHIỀU (tiếp theo)

- **Sắp xếp**

  Thuật toán bubbleSort sắp xếp tăng dần.

  Đầu vào: DaySo cần được sắp xếp

```
Duyệt từ vị trí đầu tiên i đến gần cuối trong dãy số
        Duyệt các phần tử j từ cuối đến vị trí kề sau i
                Nếu giá trị phần tử j nhỏ hơn phần tử đứng trước nó thì
                        Hoán vị hai phần tử đó
```

- **Trộn hai dãy số tăng dần và mảng kết quả cũng tăng dần**

```
DaySo C;
int i=0, j=0;
chiều dài dãy C = chiều dài dãy A + chiều dài dãy B;
Duyệt khi i<chỉ số cuối dãy số A và j<chỉ số cuối dãy số B
        Nếu A.list[i] < B.list[j] thì
                Cho A.list[i] vào dãy C
        Ngược lại
                Cho B.list[j] vào dãy C
Cuối duyệt
Duyệt khi i chưa đi hết dãy A
        Cho các phần tử chưa duyệt hết trong dãy A vào dãy C
Cuối duyệt
Duyệt khi j chưa đi hết dãy B
        Cho các phần tử chưa duyệt hết trong dãy B vào dãy C
Cuối duyệt
```

- **Thuật toán tách 1 DaySo thành 2 DaySo tại vị trí index**

```
bool tachDS(DaySo a,int index, DaySo &b, DaySo &c)
{
        Nếu index không hợp lệ
                return false;
        Ngược lại
                b.n=0;
                c.n=0;
                Duyệt i=0 tới i<index
                        Cho a.list[i] vào dãy b
                        b.n++;
                Cuối duyệt
                Duyệt i=index tới hết dãy a
                        Cho a.list[i] vào dãy c
                        c.n++;
                Cuối duyệt
                return true;
        Cuối nếu
}
```

IV. NỘI DUNG THỰC HÀNH

1/ **Chương trình mẫu: (2 điểm) Sử dụng tiếp bài lab 3 và bổ sung thêm các hàm:**

- Đọc và lưu 1 dãy số
- Chèn x vào 1 vị trí
- Xoá phần tử tại 1 vị trí
- Sắp xếp các giá trị của dãy số tăng dần.

➢ Bổ sung vào file **lab3_thuvien.h** như sau:

Dòng	Mã lệnh	Ghi chú
1	…	
2	//Nội dung file lab3_thuvien	
3	…	
4	// Bổ sung nội dung của lab4	
5	**bool** docDaySo(DaySo &ds, **char** *tenFile);	
6		
7	**bool** luuDaySo(DaySo ds, **char** *tenFile);	
8	**bool** chenXVaoViTri(DaySo &a, **int** viTri, **int** x);	
9	**bool** xoaPhanTu(DaySo &a, **int** viTri);	
10	**void** bubbleSort(DaySo &a);	

➢ Bổ sung vào file **lab3_caidat.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	…	
2	//Nội dung file lab3_chuongtrinh	
3	…	
4	// Bổ sung nội dung của lab4	
5		
6	bool docDaySo(DaySo &ds, char *tenFile)	
7	{	
8	FILE *f;	
9	f = fopen(tenFile, "rb");	
10	if (f == NULL)	
11	return false;	
12	fread(&ds, sizeof(DaySo), 1, f);	
13	fclose(f);	
14	return true;	
15	}	
16	bool luuDaySo(DaySo ds, char *tenFile)	
17	{	
18	FILE *f;	
19	f = fopen(tenFile, "wb");	
20	if (f == NULL)	
21	return false;	
22	fwrite(&ds, sizeof(DaySo), 1, f);	
23	fclose(f);	
24	return true;	
25		
26	}	
27	bool chenXVaoViTri(DaySo &a, int viTri, int x)	
28	{	
29	if (a.n == KTM || viTri < 0 || viTri > a.n)	
30	return false;	
31	for(int i=a.n; i>viTri; i--)	
32	a.arr[i] = a.arr[i-1];	
33	a.arr[viTri] = x; //chèn x vào vị trí ;	
34	a.n++;//tăng số phần tử lên 1	
35	return true;	
36	}	
37	bool xoaPhanTu(DaySo& a, int viTri)	
38	{	
39	if (viTri<0 || viTri>=a.n)	
40	return false;	
41	for(int i=viTri; i<a.n-1; i++)	

```
42          a.arr[i] = a.arr[i+1];
43      a.n--;
44      return true;
45  }
46  void bubbleSort(DaySo &a)
47  {
48      for (int i = 0; i < a.n - 1; i++)
49          for (int j = a.n - 1; j > i; j--)
50              if (a. arr[j] < a. arr[j - 1])
51                  swap(a.arr[j],a.arr[j-1]);
52
53  }
```

➢ Thay đổi hàm main trong file **lab3_chuongtrinh.cpp** như sau:

Dòng	Mã lệnh	Ghi chú

```
1   …//inchude file thư viện
2   int main()
3   {
4       DaySo x;
5       nhapDaySo(x);
6       cout<<"Day so vua nhap la: "<<endl;
7       xuatDaySo(x);
8       int so, viTri;
9       cout<<"Nhap so can them: ";
10      cin>>so;
11      cout<<"Nhap vi tri can them: ";
12      cin>>viTri;
13      if(chenXVaoViTri(x, viTri, so))
14      {
15          cout<<"Chen vao thanh cong"<<endl;
16          cout<<"Day so sau khi chen la: ";
17          xuatDaySo(x);
18      }else
19          cout<<"Chen khong duoc"<<endl;
20      cout<<"Nhap vi tri can xoa: ";
21      cin>>so;
22      if(xoaPhanTu(x, so))
23      {
24          cout<<"Xoa thanh cong"<<endl;
25          cout<<"Day so sau khi xoa la: ";
26          xuatDaySo(x);
27      }else
28          cout<<"Xoa khong duoc"<<endl;
```

```
29          bubbleSort(x);
30          cout<<"Day so sau khi sap xep la: ";
31          xuatDaySo(x);
32          return 0;
33      }
```

Yêu cầu: Sinh viên hãy bổ sung vào lab 3 và chạy chương trình trên

2/ **Bài tập vận dụng (3 điểm)**

1. Dựa vào chenXVaoViTri để xây dựng 2 hàm thêm đầu và thêm cuối.
2. Điều chỉnh hàm bubbleSort để thành hàm sắp xếp giảm dần
3. Dựa theo thuật toán đã cung cấp trong phần trên hãy xây dựng hàm trộn hai dãy số tăng dần.
4. Dựa theo thuật toán đã cung cấp trong phần trên hãy xây dựng hàm tách 1 dãy số thành 2 dãy số tại một vị trí

3/ **Bài tập tổng hợp (3 điểm)**

1. Xây dựng hàm **int timPhanTu(DaySo a, int x)** dùng để tìm vị trí phần tử đầu tiên có giá trị bằng với x
2. Xây dựng hàm kiểm tra 1 dãy số có phải là dãy số tăng không
3. Xây dựng hàm kiểm tra 1 dãy số có phần tử trùng nhau không
4. SV hãy viết hàm **đảo ngược** dãy số.
   Ví dụ :
   - Dãy số : 3 -6 8 1 9 2 1
   - Kết quả :
   - Dãy số : 1 2 9 1 8 -6 3
5. SV hãy viết hàm **xóa tất cả** phần tử có giá trị bằng x trong dãy số.
   Ví dụ :
   - Dãy số : 3 -6 8 1 9 2 1
   - Nhập x = 1
   - Kết quả → Dãy số : 3 -6 8 9 2

4/ **Bài tập làm thêm (2 điểm)**

1. Viết hàm xóa tất cả phần tử lớn nhất trong dãy số.
2. Viết hàm xóa tất cả các phần tử trùng nhau trong dãy số
3. Viết hàm sắp xếp lại dãy số sao cho các phần tử chẵn tăng dần ở đầu dãy số, các phần tử lẻ giảm dần ở cuối dãy số.
4. Viết hàm ghép dãy số 2 vào sau dãy số 1.
5. Viết hàm tách dãy số ban đầu thành 2 dãy số: dãy số 1 gồm các phần tử chẵn, dãy số 1 gồm các phần tử lẻ.
6. Viết chương trình tính các phép toán + , - , * , / cho số nguyên dài.

Gợi ý : mỗi số trong số nguyên dài ta đưa vào 1 phần tử trong mảng.

VD :

4	2	3	0	2	0	7	9	4	3	0

=> số nguyên có chiều dài 11 số đưa vào 11 phần tử của mảng.

=> các phép toán sẽ tính dựa trên 2 dãy số nguyên.

-HẾT LAB 4-

Bài 5: MẢNG 2 CHIỀU (MA TRẬN)

I. MỤC TIÊU:

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Xây dựng kiểu dữ liệu chứa ma trận có các phần tử là những số nguyên.
- Phép duyệt ma trận.
- Nhập/xuất ma trận trên thiết bị nhập/xuất chuẩn.
- Đọc/ghi ma trận (file text)
- Các phép duyệt dòng, cột của ma trận.

II. TÓM TẮT LÝ THUYẾT:

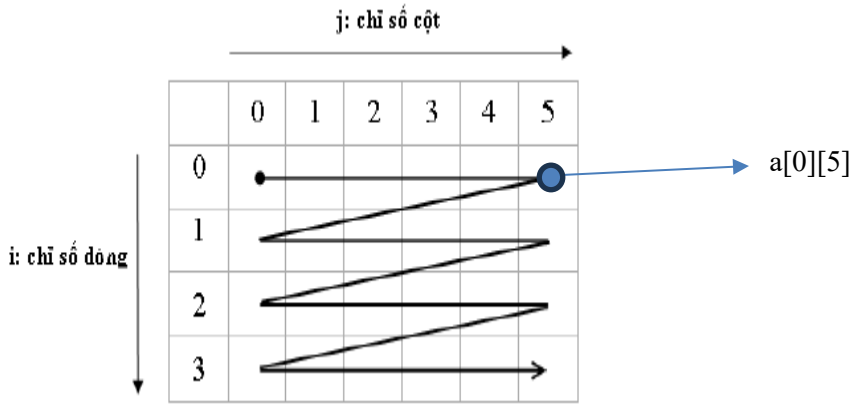
1. MẢNG 2 CHIỀU

Khai báo: <kiểu DL><Tên Mảng>[số dòng][số cột]

Ví dụ: **int a[10][20]**

Khai báo mảng a có 10 dòng, 20 cột. Có tối đa 10x20 phần tử.

Duyệt các phần tử của ma trận:



2. XÂY DỰNG KIỂU DỮ LIỆU MA TRẬN

Ma trận là mảng hai chiều có nhiều dòng và nhiều cột. Ta thấy đối với mảng hai chiều, muốn duyệt mảng phải biết được số cột và số dòng của mảng, vì vậy, để cho đơn giản, ta tạo ra kiểu dữ liệu **MaTran** có 3 thành phần là: mảng 2 chiều, số cột và số dòng.

Cài đặt:

```
const int MAX=50;//khai báo tối đa số phần tử
struct MaTran{
   int arr[MAX][MAX];
   int dong;
   int cot;
};
```

III. NỘI DUNG THỰC HÀNH

1/ Chương trình mẫu:

Khai báo cấu trúc MaTran. Viết hàm **nhập, xuất, đọc dữ liệu từ file, lưu ma trận xuống file, tính tổng, tìm phần tử lớn nhất** các phần tử của ma trận. Xây dựng hàm main để gọi các hàm trên.

➢ Cho đoạn mã trong file **lab5_thuvien.h** như sau:

Dòng	Mã lệnh	Ghi chú
1	`#define _CRT_SECURE_NO_WARNINGS`	
2	`#include<iostream>`	
3	`using namespace std;`	
4	`const int MAX = 50;`	
5	`struct MaTran{`	
6	`   int arr[MAX][MAX];`	
7	`   int dong;`	
8	`   int cot;`	
9	`};`	
10	`void nhapMaTran (MaTran& a);`	
11	`void xuatMaTran (MaTran a);`	
12	`bool docMaTran(MaTran &a, char *fileName);`	
13	`bool luuMaTran(MaTran a, char *fileName);`	
14	`int tong(MaTran a);`	
15	`int demDuong(MaTran a);`	
16	`int max(MaTran a);`	

➢ Cho đoạn mã trong file **lab5_caidat.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	`#include "lab5_thuvien.h"`	
2	`void nhapMaTran (MaTran& a)`	
3	`{`	
4	`  cout<<"\n Nhap so dong : ";`	
5	`  cin>>a.dong;`	
6	`  cout<<"\n Nhap so cot : ";`	
7	`  cin>>a.cot;`	
8	`  for(int i=0; i<a.dong; i++)`	
9	`    for(int j=0; j<a.cot; j++){`	
10	`      cout << "\n arr[" << i << "][" << j << "]= ";`	
11	`      cin >> a.arr[i][j];`	
12	`    }`	
13	`}`	
14	`void xuatMaTran (MaTran a)`	
15	`{`	
16	`  for(int i=0; i<a.dong; i++){`	
17	`    cout<<endl;`	
18	`    for(int j=0; j<a.cot; j++)`	
19	`      cout<<"\t"<<a.arr[i][j];`	
20	`  }`	
21	`}`	
22	`bool docMaTran(MaTran &a, char *fileName){`	
23	`  FILE *f;`	
24	`  f = fopen(fileName, "r");`	
25	`  if (f == NULL)`	
26	`    return false;`	
27	`  fscanf(f, "%d",&a.dong);`	
28	`  fscanf(f, "%d",&a.cot);`	
29	`  for (int i=0; i<a.dong; i++)`	
30	`    for (int j=0; j<a.cot; j++)`	
31	`      fscanf(f, "%d",&a.arr[i][j]);`	
32	`  fclose(f);`	
33	`  return true;`	
34	`}`	
35	`bool luuMaTran(MaTran a, char *fileName){`	
36	`  FILE *f;`	
37	`  f = fopen(fileName, "w");`	
38	`  if (f == NULL)`	
39	`    return false;`	
40	`  fprintf(f, "%d\t",a.dong);`	
41	`  fprintf(f, "%d\n",a.cot);`	

```
42      for (int i=0; i<a.dong; i++)
43          for (int j=0; j<a.cot; j++)
44              fprintf(f, "%d\t",a.arr[i][j]);
45      fclose(f);
46      return true;
47  }
48  int tong(MaTran a)
49  {
50      int s=0;
51      for(int i=0; i<a.dong; i++)
52          for(int j=0; j<a.cot; j++)
53              s+=a.arr[i][j];
54      return s;
55  }
56  int demDuong(MaTran a)
57  {
58      int dem=0;
59      for(int i=0; i<a.dong; i++)
60          for(int j=0; j<a.cot; j++)
61              if(a.arr[i][j]>0)
62                  dem++;
63      return dem;
64  }
65  int max(MaTran a)
66  {
67      int m=a.arr[0][0];
68      for (int i=0; i<a.dong; i++)
69          for (int j=0; j<a.cot; j++)
70              if(a.arr[i][j]>m)
71                  m=a.arr[i][j];
72      return m;
    }
```

➢ Cho đoạn mã trong file **lab5_chuongtrinh.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	`#include "lab5_thuvien.h"`	
2	`int main()`	
3	`{`	
4	`    MaTran a;`	
5	`    nhapMaTran(a);`	
6	`    //docMaTran(a, "matran.txt");`	
7	`    cout<<"Ma tran vua nhap la: ";`	
8	`    xuatMaTran(a);`	

```
9         cout<<"\nTong cac gia tri trong mang la: ";
10        cout<<tong(a)<<endl;
11                cout<<"Gia tri lon nhat la: "<<max(a)<<endl;
12        if (luuMaTran(a, "matran.txt"))
13           cout<<"Luu thanh cong";
14        else
15           cout<<"Luu khong thanh cong";
16        cout<<endl;
17        return 0;
18     }
```

Yêu cầu: **(2 điểm)**

- Sinh viên hãy nhập và chạy chương trình trên
- Sửa lại hàm main để đọc file vừa lưu
- Thêm vào hàm main để gọi được hàm **demDuong**.

2/ Bài tập vận dụng (3 điểm)

1. Sửa hàm **tong** thành hàm **tongChan**, để tính tổng các phần tử **chẵn** có trong ma trận.
2. Sửa hàm **demDuong** thành hàm dem để đếm các số chính phương có trong ma trận
3. Hãy vận dụng hàm **max** để viết hàm tìm số nguyên tố lớn nhất có trong ma trận. Nếu không có số nguyên tố nào thì hàm trả về 0.

3/ Bài tập tổng hợp (3điểm)

1. Viết hàm tính tổng các dòng có trong 1 ma trận

Hướng dẫn:

3	2	5
1	7	-4
9	-8	11

- Dòng 1 tổng = 10
- Dòng 2 tổng = 4
- Dòng 3 tổng = 12

2. Tìm dòng có tổng lớn nhất. Ví dụ ở trên dòng có tổng lớn nhất là dòng.
3. Viết hàm tính tổng các phần tử lớn nhất trên mỗi dòng.
4. Viết hàm tính tổng các cột có trong ma trận.
5. Viết hàm hoán đổi giá trị 2 cột.

4/ Bài tập làm thêm (2 điểm)

1. Viết hàm sắp xếp các phần tử tăng dần trên từng cột
2. Viết hàm sắp xếp các phần tử giảm dần trên dòng

3. Viết hàm tính tổng 2 ma trận
4. Viết hàm tính tích 2 ma trận

-HẾT LAB 5-

Bài 6: CÁC PHÉP TOÁN TRÊN MA TRẬN VUÔNG

I. MỤC TIÊU:

Sau khi thực hành xong bài này, sinh viên phải nắm được:

- Đọc/ghi file
- Thực hiện các thao tác trên ma trận vuông (duyệt đường chéo chính, đường chéo phụ, các phép tình có trên ma trận vuông, …)
- Thuật toán cho lượng từ tồn tại và với mọi

II. TÓM TẮT LÝ THUYẾT:

1. Ma Trận vuông

Nội dung	Ví dụ
Khái niệm: - Ma trận vuông là trường hợp đặc biệt của ma trận có **số dòng = số cột**. Ta có thể xây dựng kiểu MaTranVuong gồm 2 thành phần gồm: mảng 2 chiều, cấp của ma trận vuông.	- Ma trận vuông cấp 3, có tối đa 3x3 phần tử 5 4 8 -9 12 1 9 -12 15
Khai báo: **<kiểu DL> <Tên Mảng>[số dòng][số cột]**	**int a[10][10]** – Khai báo ma trận vuông A có 10 dòng, 10 cột. Có tối đa 10x10 phần tử. **float b[5][5]** - Khai báo ma trận vuông B có 5 dòng, 5 cột. Gọi là ma trận vuông cấp 5. Có tối đa 5x5 phần tử.

2. Xây dựng kiểu ma trận

Ý tưởng: Ta thấy đối với ma trận vuông, muốn duyệt ma trận phải biết được cấp của ma trận. Vì vậy, để cho đơn giản, ta tạo ra kiểu dữ liệu **MaTranVuong** có 2 thành phần là mảng 2 chiều, cấp của ma trận.

Cài đặt:

```
const int MAX = 50; //khai báo tối đa số phần tử
struct MaTranVuong{
        int list[MAX][MAX];
        int n; //cấp của ma trận
};
```

Duyệt các phần tử trên đường chéo chính: Các phần tử trên đường chéo chính có chỉ số dòng bằng chỉ số cột: a[i][i]	**Cài đặt:** for(int i=0;i<a.n;i++) //Duyệt phần tử a.list[i][i]
- **Duyệt các phần tử trên đường chéo phụ** Các phần tử trên đường chéo phụ có chỉ số dòng là i và chỉ số cột là n – i -1: a[i][n-i-1]	for (int i=0;i<a.n;i++) //Duyệt phần tử a.list[i][a.n-i-1]

- **Lượng từ tồn tại và với mọi**

Một trong những thủ thuật đóng vai trò quan trọng trong lập trình là làm sao cài đặt việc kiểm tra các khẳng định có chứa lượng từ tồn tại hay với mọi. Chẳng hạn khẳng định một số nguyên n>1 không phải là số nguyên tố được viết bởi mệnh đề: "Tồn tại số tự nhiên m sao cho 1<m<n và m là ước của n". Sau đây là thuật toán tổng quát cho loại mệnh đề này (Trong đó P(x) là một khẳng định tùy thuộc và một biến x nào đó):

- **Cài đặt kiểm tra tồn tại:** **($\exists x \in S$) (P(x))**	KT := false Duyệt x ∈ S Nếu P(x)thỏa thì KT := true Dừng quá trình duyệt S Cuối nếu Cuối duyệt
- **Cài đặt kiểm tra với mọi** **∀ ($\forall x \in S$) (P(x))**	KT := true Duyệt x ∈ S Nếu not P(x) thì KT := false Dừng quá trình duyệt S Cuối nếu Cuối duyệt

- Cả hai thuật toán trên: giá trị của biến KT khi thuật toán dừng, chính là chân trị của mệnh đề cần kiểm tra, giá trị này thường được cài đặt như giá trị trả về của hàm khi sử dụng một ngôn ngữ lập trình cụ thể.

III. NỘI DUNG THỰC HÀNH

1/ Chương trình mẫu (2 điểm)

Câu 1: Đoạn mã sau, dùng để phát sinh, đọc, ghi và xuất ra một ma trận vuông. Có 2 hàm: **xuatCheoChinh** dùng để xuất các phần tử có trên đường chéo chính và hàm **tinhTongCheoPhu** dùng để tính tổng các phần tử có trên đường chéo phụ.

➢ Cho đoạn mã trong file **lab6_thuvien.h** như sau:

Dòng	Mã lệnh	Ghi chú
1	`#include<iostream>`	
2	`#include "time.h"`	
3	`using namespace std;`	
4	`const int KTMT=50;`	
5	`struct MaTranVuong{`	
6	`  int mt[KTMT][KTMT];`	
7	`  int bac; //dong=cot=bac`	
8	`};`	
9	`void phatSinhMaTran(MaTranVuong &mtv);`	
10	`void xuatMaTran(MaTranVuong mtv);`	
11	`bool docMaTran(MaTranVuong &mtv, char *fileName);`	
12	`bool ghiMaTran(MaTranVuong mtv, char *fileName);`	
13	`void xuatCheoChinh(MaTranVuong mtv);`	
14	`int tinhTongCheoPhu(MaTranVuong mtv);`	

➢ Cho đoạn mã trong file **lab6_caidat.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	`#include"lab6_thuvien.h"`	
2	`void phatSinhMaTran(MaTranVuong &mtv)`	
3	`{`	
4	`  srand(time(0));`	
5	`  mtv.bac=3+rand()%3;`	
6	`  for (int i=0; i<mtv.bac; i++)`	
7	`  for (int j=0; j<mtv.bac; j++)`	
8	`  mtv.mt[i][j]=rand()%10-5;`	
9	`}`	
10	`void xuatMaTran(MaTranVuong mtv)`	
11	`{`	
12	`  for (int i=0; i<mtv.bac; i++){`	
13	`  for (int j=0; j<mtv.bac; j++)`	
14	`  cout<<mtv.mt[i][j]<<"\t";`	
15	`  cout<<endl;`	
16	`  }`	

```
17      }
18      bool docMaTran(MaTranVuong &mtv, char *fileName){
19          FILE *f;
20          f = fopen(fileName, "rb");
21          if (f == NULL)
22              return false;
23          fread(&mtv, sizeof(MaTranVuong), 1, f);
24          fclose(f);
25          return true;
26      }
27      void xuatMaTran(MaTranVuong mtv)
28      {
29          for (int i=0; i<mtv.bac; i++){
30              for (int j=0; j<mtv.bac; j++)
31                  cout<<mtv.mt[i][j]<<"\t";
32              cout<<endl;
33          }
34      }
35      void xuatCheoChinh(MaTranVuong mtv)
36      {
37          for (int i = 0; i < mtv.bac; i++)
38                  cout << mtv.mt[i][i] << "\t";
39      }
40      int tinhTongCheoPhu(MaTranVuong mtv){
41          int s = 0;
42          for (int i = 0; i < mtv.bac; i++)
43                  s += mtv.mt[i][a.n - i - 1];
44          return s;
45      }
```

➢ Cho đoạn mã trong file **lab6_chuongtrinh.cpp** như sau:

Dòng	Mã lệnh	Ghi chú

```
1   #include"lab6_thuvien.h"
2   int main()
3   {
4       MaTranVuong m;
5       phatSinhMaTran(m);
6       cout<<"Ma tran vua phat sinh la: "<<endl;
7       xuatMaTran(m);
8       if (ghiMaTran(m, "mtv.dat")) {
9           cout<<"Luu thanh cong"<<endl;
10      }else
11          cout<<"Co loi khi luu"<<endl;
```

```
12          system("pause");
13      return 0;
14  }
```

Câu 2: Đoạn mã sau có 2 hàm, hàm **bool kiemTraLe** dùng để kiểm tra mọi phần tử có trong ma trận toàn là số lẻ hay không, nếu có trả về true, nếu không trả về false. Hàm **bool kiemTraBoi3** dùng để kiểm tra có tồn tại phần tử là bội của 3 trên đường chéo phụ hay không, nếu tồn tại thì trả về true, ngược lại trả về false.

➢ Trong file **lab6_thuvien.h Câu 1** bổ sung đoạn mã như sau:

Dòng	Mã lệnh	Ghi chú
1	…	
2	bool kiemTraLe(MaTranVuong a);	
3	bool kiemTraBoi3(MaTranVuong a);	

➢ Trong file **lab6_caidat.cpp Câu 1** bổ sung đoạn mã như sau:

Dòng	Mã lệnh	Ghi chú
1	…	
2	bool kiemTraLe(MaTranVuong a){	
3	bool kt = true;	
4	for (int i = 0; i < a.bac; i++)	
5	for (int j = 0; j < a.bac; j++)	
6	if (a.mt[i][j] % 2 == 0)	
7	kt = false;	
8	return kt;	
9	}	
10	bool kiemTraBoi3(MaTranVuong a){	
11	bool kt = false;	
12	for (int i = 0; i < a.bac; i++)	
13	if (a.mt[i][a.bac - i - 1] % 3 == 0)	
14	kt = true;	
15	return kt;	
16	}	

Câu 3: Cho đoạn mã dùng để in tam giác trái dưới như sau:

Dòng	Mã lệnh	Ghi chú
1	void inTamGiacTraiDuoi(MaTranVuong a){	
2	for (int i = 0; i < a.bac; i++){	

```
3                   for (int j = 0; j < a.bac; j++)
4                       if (j <= i)
5                           cout << a.mt[i][j] << "\t";
6                   cout << endl;
7               }
8   }
```

Yêu cầu: (2 điểm)

- Sinh viên hãy nhập và chạy chương trình trên
- Hãy bổ sung vào hàm main để chạy được các hàm trong câu 2 và câu 3

2/ Bài tập vận dụng (3 điểm)

1. Yêu cầu viết hàm in tam giác trên trái

Ví dụ: Cho ma trận vuông cấp 3 như sau:

1	2	3
4	5	6
7	8	9



1	2	3
4	5	
7		

2. Bổ sung vào chương trình, yêu cầu viết hàm in tam giác phía trên bên phải

Ví dụ:

1	2	3
4	5	6
7	8	9



1	2	3
	5	6
		9

3. Bổ sung vào chương trình, yêu cầu viết hàm in tam giác phía dưới bên phải

Ví dụ:

1	2	3
4	5	6
7	8	9



		3
	5	6
7	8	9

4. Viết hàm kiểm tra xem ở trong tam giác phía dưới bên phải, tồn tại phần tử là số âm hay không? Nếu có trả về true, ngược lại trả về false.

5. Viết hàm kiểm tra xem ở tam giác phải trên tất cả các phần tử là số chẵn hay không? Nếu tất cả là số chẵn thì trả về true, ngược lại trả về false.

3/ Bài tập tổng hợp (3 điểm)

1. Viết hàm nhập thêm vào ma trận thứ 2. Viết hàm tính tổng hai ma trận.

1	2	3
4	5	6
7	8	9

+

-5	9	3
12	15	-6
17	20	14

=

-4	11	6
16	20	0
24	28	23

2. Viết hàm tìm ma trận chuyển vị của một MaTranVuong.

Ví dụ:

1	2	3
4	5	6
7	8	9



1	4	7
2	5	8
3	6	9

3. Viết hàm chuyển đổi các phần tử qua đường chéo phụ

Ví dụ:

1	2	3
4	5	6
7	8	9



9	6	7
8	5	2
3	4	1

4/ Bài tập tổng hợp về nhà. (2 điểm)

1. Viết hàm tìm ma trận chuyển vị của một MaTranVuong
2. Viết hàm tính tích 2 MaTranVuong.
3. Viết hàm hoán vị hai đường chéo
4. Viết hàm sắp xếp MaTranVuong tăng dần theo dòng và theo cột
5. Viết hàm kiểm tra xem 1 MaTranVuong có đối xứng qua đường chéo chính không?

-HẾT LAB 6-

Bài 7: ĐỆ QUY

I. MỤC TIÊU:

Sau khi thực hành xong bài này, sinh viên có thể :

- Biết cách lập trình đệ quy.
- Ứng dụng đệ quy cài đặt thuật toán QuickSort và MergeSort.
- Cài đặt thuật toán tìm kiếm nhị phân trên dãy số đã được sắp xếp.

II. TÓM TẮT LÝ THUYẾT:

1) Đệ quy:

- Đệ quy (Recursion) là một trong những giải thuật khá quen thuộc trong lập trình mở rộng ra trong toán học (thường được gọi là quy nạp)

  VD: n! = n * (n – 1)! (n>=0)

- Một đối tượng được mô tả thông qua chính nó được gọi là mô tả đệ quy.

  VD: Một node trong cây nhị phân chứa thông tin của node và hai node con của nó cũng là một node.

```
struct Node
{
   int info;
   Node *pLeft;
   Node *pRight;
   };
```

- **Hàm đệ quy:**

  Trong lập trình, một hàm được gọi là đệ quy khi nó gọi chính nó trong thân hàm.

  VD:

```
//neu n<0 khong tinh duoc giai thua => ham se tra ve 0
long giaiThua(int n)
{
   if(n<0)
      return 0;
   if(n==0 || n==1)
      return 1;
   return giaiThua(n-1)*n;

}
```

Hàm đệ quy gồm 2 phần:

- Phần cơ sở: Điều kiện thoát khỏi đệ quy.
- Phần đệ quy: Thân hàm có chứa lời gọi đệ quy.

▪ **Thiết kế giải thuật đệ quy:**

- Xác định tham số bài toán.
- Phân tích trường hợp chung: Đưa bài toán về bài toán nhỏ hơn cùng loại, dần dần tiến tới trường hợp suy biến.
- Tìm trường hợp suy biến (điều kiện thoát khỏi đệ quy).

2) Merge Sort (Sắp xếp trộn):

Merge sort là một thuật toán chia để trị. Thuật toán này chia mảng cần sắp xếp thành 2 nửa. Tiếp tục lặp lại việc này ở các nửa mảng đã chia. Sau cùng gộp các nửa đó thành mảng đã sắp xếp. Hàm merge() được sử dụng để gộp hai nửa mảng. Hàm merge(arr, l, m, r) là tiến trình quan trọng nhất sẽ gộp hai nửa mảng thành 1 mảng sắp xếp, các nửa mảng là arr[l…m] và arr[m+1…r] sau khi gộp sẽ thành một mảng duy nhất đã sắp xếp.

▪ **Thuật toán merge sort:**

mergeSort(int a[], int l, int r)

Nếu r > l

1. Tìm chỉ số nằm giữa mảng để chia mảng thành 2 nửa:
   m = (l+r)/2
2. Gọi đệ quy hàm mergeSort cho nửa đầu tiên:
   mergeSort(a, l, m)
3. Gọi đệ quy hàm mergeSort cho nửa thứ hai:
   mergeSort(a, m+1, r)
4. Gộp 2 nửa mảng đã sắp xếp ở (2) và (3):
   merge(a, l, m, r)

merge(int a[], int l, int m, int r)

1. Chép các phần tử ra các mảng tạm (mảng L từ l đến m, mảng R từ m+1 đến r)
2. Trộn hai mảng tạm vào lại mảng a sao cho các phần tử đứng đúng thứ tự.
3. Chép các phần tử còn lại sau khi trộn (nếu có) của hai mảng.

3) Quick Sort (Sắp xếp nhanh):

Thuật toán sắp xếp quick sort là một thuật toán chia để trị. Nó chọn một phần tử trong mảng làm điểm đánh dấu (pivot). Thuật toán sẽ thực hiện chia mảng thành các mảng con dựa vào pivot đã chọn. Việc lựa chọn pivot ảnh hưởng rất nhiều tới tốc độ sắp xếp. Dưới đây là một số cách để chọn pivot thường được sử dụng:

1. Luôn chọn phần tử đầu tiên của mảng.
2. Luôn chọn phần tử cuối cùng của mảng. *(Được sử dụng trong bài lab này)*
3. Chọn một phần tử ngẫu nhiên.
4. Chọn một phần tử có giá trị nằm giữa mảng.

Mấu chốt chính của thuật toán quick sort là việc phân hoạch dãy số (Xem hàm partition()). Cho một mảng và một phần tử x là pivot. Đặt x vào đúng vị trí của mảng đã sắp xếp. Di chuyển tất cả các phần tử của mảng nhỏ hơn x sang bên trái vị trí của x, và di chuyển tất cả các phần tử của mảng mà lớn hơn x sang bên phải vị trí của x.

- **Thuật toán phân hoạch trong quick sort:**
  - Đặt pivot là phần tử cuối cùng của dãy số a.
  - Đặt phần tử trái nhất của dãy số có chỉ số là left, phần tử phải nhất của dãy số có chỉ số là right -1 (bỏ qua phần tử pivot).
  - Trong khi left < right mà a[left] > pivot và a[right] < pivot thì đổi chỗ hai phần tử left và right.
  - Sau cùng, ta đổi chỗ hai phần tử left và pivot cho nhau. Khi đó, phần tử left đã đứng đúng vị trí và chia dãy số làm hai (bên trái và bên phải)

4) Tìm kiếm nhị phân (Binary Search)

Cho mảng a đã sắp xếp tăng dần, việc tìm kiếm phần tử x có thể thực hiện như sau:

1. Xét đoạn mảng a[left…right] cần tìm kiếm phần tử x. Ta so sánh x với phần tử ở vị trí giữa của mảng (mid = (left + right)/2).
2. Nếu phần tử a[mid] = x. Kết luận và thoát chương trình.
3. Nếu a[mid] < x. Chỉ thực hiện tìm kiếm trên đoạn arr[mid+1…right].
4. Nếu a[mid] > x. Chỉ thực hiện tìm kiếm trên đoạn arr[left…mid-1]

III. NỘI DUNG THỰC HÀNH:

1/ Chương trình mẫu: (4 điểm)

Câu 1: Mô tả chương trình:

- Nhập n nguyên dương. In ra n giai thừa. In ra phần tử thứ n của dãy Fibonacci.
- Sắp thứ tự dãy A bằng thuật toán MergeSort. In ra dãy A đã sắp tăng dần.
- Sắp thứ tự dãy B bằng thuật toán QuickSort. In ra dãy B đã sắp tăng dần.
- Nhập giá x cần tìm trong dãy A. Dùng thuật toán Tìm kiếm nhị phân tìm vị trí của x có trong dãy A.

➢ Cho đoạn mã trong file **lab7_thuvien.h** như sau:

Dòng	Mã lệnh	Ghi chú
1	`#include <iostream>`	
2	`#include <iomanip>`	
3	`using namespace std;`	
4	`const int MAX = 50;`	
5	`struct DaySo`	
6	`{`	
7	`    int mang[MAX];`	
8	`    int soPhanTu;`	
9	`};`	
10	`long giaiThua(int n);`	
11	`int fibonacci(int n);`	
12	`int binarySearch(int a[], int l, int r, int x);`	
13	`void mergeSort(int a[], int l, int r);`	
14	`void merge(int a[], int l, int m, int r);`	
15	`void quickSort(int a[], int l, int h);`	
16	`int partition(int a[], int low, int high);`	

➢ Cho đoạn mã trong file **lab7_caidat.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	//thêm #include	
2	//Định nghĩa hàm	
3	//neu n<0 khong tinh duoc giai thua => ham se tra ve 0	
4	long giaiThua(int n)	
5	{	
6	if (n < 0)	
7	return 0;	
8	if (n == 0 || n == 1)	
9	return 1;	
10	return giaiThua(n - 1) * n;	
11	}	
12	//Ham xac dinh so Fibonacci thu n. Day Fibonacci 1 1 2 3 5 8 ...	
13	//Neu n < 1 (khong hop ly) ham se tra ve 0	
14	int fibonacci(int n)	
15	{	
16	if (n < 1)	
17	return 0; //khong thuoc day so Fibonacci	
18	if (n < 3)	
19	return 1;	
20	return fibonacci(n - 1) + fibonacci(n - 2);	
21	}	
22		
23	int binarySearch(int a[], int l, int r, int x)	
24	{	
25	if (l > r)	
26	return -1;	
27	int mid = (l + r) / 2;	
28	if (a[mid] == x) //truong hop tim thay	
29	return mid;	
30	if (a[mid] > x)	
31	return binarySearch(a, l, mid - 1, x); //tim x trong nua mang	
32	ben trai	
33	return binarySearch(a, mid + 1, r, x); //tim x trong nua mang	
34	ben phai	
35	}	
36	//l la chi so trai, r la chi so phai cua mang	
37	void mergeSort(int a[], int l, int r)	
38	{	
39	if (l >= r)	
40	return;	

```
41          int mid=(l + r) / 2;
42          //Gọi đệ quy tiếp tục chia đôi từng nửa mảng
43          mergeSort(a, l, mid);
44          mergeSort(a, mid + 1, r);
45          merge(a, l, mid, r);
46      }
47      //Gop 2 mang con a[1..mid] va a[mid+1..r]
48      void merge(int a[], int l, int m, int r)
49      {
50          int i, j, k;
51          int n1 = m - l + 1;
52          int n2 = r - m;
53          //Tao cac mang tam
54          int *lArr = new int[n1];
55          int* rArr = new int[n2];
56          //Copy du lieu sang cac mang tam
57          for (i = 0; i < n1; i++)
58              lArr[i] = a[l + i];
59          for (i = 0; i < n2; i++)
60              rArr[i] = a[m + 1 + i];
61          //Gop hai mang tam vao mang a
62          i = 0;
63          j = 0;
64          k = l;
65          while (i < n1 && j < n2)
66          {
67              if (lArr[i] <= rArr[j])
68              {
69                  a[k] = lArr[i];
70                  i++;
71              }else{
72                  a[k] = rArr[j];
73                  j++;
74              }
75              k++;
76          }
77      //copy cac phan tu con lai cua mang lArr vao a neu co
78          while (i < n1)
79          {
80              a[k] = lArr[i];
81              i++;
82              k++;
83          }
```

```
84  //copy cac phan tu con lai cua mang rArr vao a neu co
85      while (j < n2)
86      {
87          a[k] = rArr[j];
88          j++;
89          k++;
90      }
91        //Xoa 2 mang tam
92      delete []lArr;
93      delete []rArr;
94  }
95  void quickSort(int a[], int l, int h)
96  {
97      if (l >= h)
98          return;
99  //pi la vi tri cua phan tu chia mang lam 2 mang trai va phai
100     int pi = partition(a, l, h);
101     quickSort(a, l, pi-1);
102     quickSort(a, pi + 1, h);
103 }
104 int partition(int a[], int low, int high)
105 {
106     int pivot = a[high];
107     int left = low;
108     int right = high - 1;
109     while (true)
110     {
111         while (left <= right && a[left] < pivot)
112             left++;
113         while (right >= left && a[right] > pivot)
114             right--;
115         if (left >= right)
116             break;
117         swap(a[left], a[right]);
118         left++;
119         right--;
120     }
121     swap(a[left], a[high]);
122     return left;
123 }
124
```

➢ Cho đoạn mã trong file **lab7_chuongtrinh.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	//thêm include …	
2	int main()	
3	{	
4	int a[MAX];	
5	int n, x;	
6	do {	
7	cout << "Nhap so phan tu: ";	
8	cin >> n;	
9	} while (n <= 0);	
10	for (int i = 0; i < n; i++)	
11	{	
12	cout << "Nhap a[" << i << "]: ";	
13	cin >> a[i];	
14	}	
15	cout << n << "! = " << giaiThua(n) << endl;	
16	cout << "So Fibonacci thi " << n << " la " << fibonacci(n) <<	
17	endl;	
18	mergeSort(a, 0, n-1);	
19	for (int i = 0; i < 6; i++)	
20	cout << a[i] << " ";	
21	cout << endl;	
22	cout << "Ban muon tim so may? ";	
23	cin >> x;	
24	int kq = binarySearch(a, 0, 5, x);	
25	if (kq >= 0)	
26	cout << "Tim thay tai vi tri " << kq << endl;	
27	else	
28	cout << "Khong tim thay"<<endl;	
29	return 0;	
30	}	

Yêu cầu sinh viên: (2 điểm)

a) Biên dịch chương trình trên.

b) Cho biết kết quả in ra màn hình khi người dùng nhập dữ liệu sau:

n = 5, x = 8

n = 7, x =10

c) Chạy từng bước hàm **giaiThua** với n = 5.

d) Chạy từng bước hàm **fibonaci** với n = 7

Câu 2: Sinh viên bổ sung các đoạn code bên dưới để tiến hành thực hiện các thao tác cơ bản (khai báo, tạo dữ liệu, lưu, đọc file, in danh sách) trên **mảng 1 chiều có các phần tử là kiểu cấu trúc**.

➢ Trong file **lab7_thuvien.h** bổ sung đoạn mã sau:

Dòng	Mã lệnh	Ghi chú
1	…	
2	struct SinhVien	
3	{	
4	char maSV[5];	
5	char hoLot[30];	
6	char ten[10];	
7	float diemTrungBinh;	
8	};	
9	struct DanhSachSV	
10	{	
11	int siSo;	
12	SinhVien mang[MAX];	
13	};	
14	DanhSachSV taoDuLieu();	
15	DanhSachSV loadDuLieu(char *tenFile);	
16	bool luuDanhSachSV(DanhSachSV ds, char* tenFile);	
17	void inSinhVien(SinhVien sv);	
18	void inDanhSachSV(DanhSachSV dssv);	
19	//	
20	//các hàm bên dưới chưa có định nghĩa hàm, SV xây dựng các	
21	//định nghĩa hàm theo yêu cầu trong bài tập tổng hợp	
22	void merge(SinhVien a[], int l, int m, int r);	
23	void mergeSort(SinhVien a[], int l, int r);	
24	void quickSort(SinhVien a[], int l, int h);	
25	int partition(SinhVien a[], int low, int high);	
26	int binarySearch(SinhVien a[], int l, int r, char* maSV);	

➢ Trong file **lab7_caidat.cpp** bổ sung đoạn mã sau:

Dòng	Mã lệnh	Ghi chú
1	…	
2	DanhSachSV taoDuLieu()	
3	{	
4	DanhSachSV ds = {10,{	
5	{"SV01","Le Thi","Be",7.7},	
6	{"SV03","Nguyen Ngoc","Ti",4.5},	
7	{"SV06","Le Van","Phuoc",8.0},	
8	{"SV04","Tran Thi","Binh",6.0},	

```
9                   {"SV05","Ngo Bao","Chau",7.0},
10                  {"SV07","Le Thanh","Tam",9.0},
11                  {"SV02","Le Thi Hong","Gam",3.5},
12                  {"SV08","Nguyen Duy","An",8.5}
13                  }
14          };
15          return ds;
16  }
17  DanhSachSV loadDuLieu(char* tenFile)
18  {
19          DanhSachSV ds;
20          ds.siSo = 0;
21          FILE* file;
22          errno_t file_in;
23          //mo file de doc
24          file_in = fopen_s(&file, tenFile, "rb");
25          if (file == NULL)
26          {
27                  cout << "Loi doc file!";
28                  return ds;
29          }
30          int i = 0;
31          fread(&ds, sizeof(DanhSachSV), 1, file);
32
33          // dong file
34          fclose(file);
35          return ds;
36  }
37  bool luuDanhSachSV(DanhSachSV ds,char* tenFile)
38  {
39          FILE* file;
40          errno_t file_in;
41          //mo file de doc
42          file_in = fopen_s(&file, tenFile, "wb");
43          if (file == NULL)
44          {
45                  cout << "Loi doc file!";
46                  return false;
47          }
48          fwrite(&ds, sizeof(DanhSachSV), 1, file);
49          fclose(file);
50          return true;
51  }
```

➢ Trong file **lab7_chuongtrinh.cpp** bổ sung đoạn mã sau:

Dòng	Mã lệnh	Ghi chú
1	int main()	
2	{	
3	…	
4	char* ten = new char[50];	
5	strcpy_s(ten,50, "dssv.dat");	
6	//DanhSachSV ds=taoDuLieu();	
7	//luuDanhSachSV(ds, ten);	
8	DanhSachSV ds;	
9	ds = loadDuLieu(ten);	
10	inDanhSachSV(ds);	
11	return 0;	
12	}	

2/ Bài tập vận dụng (3 điểm)

1. Thay đổi hàm mergeSort để sắp xếp mảng giảm dần.
2. Thay đổi hàm quickSort để sắp xếp mảng giảm dần.
3. Thay đổi hàm binarySearch để tìm kiếm trên mảng có thứ tự giảm dần.

3/ Bài tập tổng hợp: (2 điểm)

1. Cài đặt hàm mergeSort để sắp danh sách sinh viên tăng dần theo MSSV.
2. Cài đặt hàm tìm binarySearch theo MSSV.
3. Cài đặt hàm quickSort để sắp danh sách sinh viên tăng dần theo tên, nếu trùng tên thì sắp tăng dần theo họ lót.

4/ Bài tập làm thêm: (3 điểm)

Dùng đệ quy để giải các bài toán sau:

1. Tìm chữ số lớn nhất của số nguyên dương n.
2. Tìm chữ số nhỏ nhất của số nguyên dương n.
3. Kiểm tra số nguyên dương n có toàn chữ số chẵn hay không?
4. Kiểm tra số nguyên dương n có toàn chữ số lẻ hay không?
5. Cho số nguyên dương n, tìm chữ số đầu tiên của n.

-HẾT LAB 7-

Bài 8: CHUỖI KÝ TỰ

I. MỤC TIÊU:

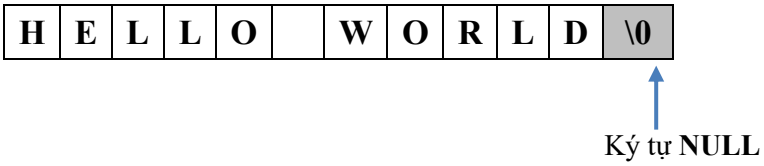
Sau khi thực hành xong bài này, sinh viên có thể :

- Biết các sử dụng các hàm xử lý chuỗi cơ bản (strcat, strcpy, strlen, strcmp, strchr, strstr, tolower, toupper, …).
- Ứng dụng cho các bài toán xử lý chuỗi.

II. TÓM TẮT LÝ THUYẾT:

1) Chuỗi:

- Chuỗi là mảng 1 chiều gồm có các phần tử là kiểu char và được kết thúc bởi ký tự **NULL** (‘\0’)



- **Khai báo:**

  char <ten_chuoi>[số ký tự + 1];

- **Truy cập các phần tử trong chuỗi:**

  <ten_chuoi>[chỉ số] (từ 0 đến chiều dài chuỗi -1)

- **Nhập / Xuất chuỗi:**
  - Nhập chuỗi: dùng **cin.getline**(<ten_chuoi>, <số ký tự + 1>);
  - Bỏ qua ký tự trên dòng nhập: **cin.ignore**(<số ký tự>). Lệnh này thường được sử dụng để bỏ qua ký tự Enter trên dòng nhập.

2) Các hàm xử lý chuỗi cơ bản

- Hàm **strlen()**: trả về số kí tự có trong chuỗi.

  int strlen(char *s)

  Ví dụ: strlen("Hello") sẽ trả về giá trị là 5.

- Hàm **strcat()**: nối chuỗi s2 vào cuối chuỗi s1

  char *strcat(char *s1, char *s2)

  Ví dụ:

  char dest[50] = "Mon hoc ";

  char src[50] = "ky thuat lap trinh";

  strcat(dest, src); // dest sẽ có giá trị là “Mon hoc ky thuat lap trinh”

- Hàm **strcpy():** sao chép nội dung chuỗi s2 vào s1

  char *strcpy(char *s1, char*s2)

  Ví dụ:

  ```
  char dest[50] = "Mon hoc ";
  char src[50] = "ky thuat lap trinh";
  strcpy(dest, src);// dest sẽ có gía trị là "ky thuat lap trinh"
  ```

- Hàm **strcmp()**: so sánh 2 chuỗi; kết quả trả về <0; =0; >0

  int strcmp(char * s1, char *s2)

  Ví dụ:

  ```
  char s1[50] = "Abc";
  char s2[50] = "abc";
  cout<<strcmp(s1, s2)<<endl; //xuất ra 1 số âm vì 'A' < 'a' => s1<s2
  ```

- Hàm **strchr()**: tìm ký tự *ch* trong chuỗi *s*, nếu không tìm thấy trả về NULL.

  ```
  char * strchr(char *s, int ch)
  ```

  Ví dụ:

  ```
  char s1[50] = "mon hoc nhap mon lap trinh";
  cout<<strchr(s1, 'o')<<endl; // xuất ra chuỗi "on hoc nhap mon lap trinh"
  ```

- Hàm **strstr**(): tìm vị trí đầu tiên của chuỗi con *sub* trong chuỗi *s*, nếu không tìm được trả về NULL.

  char* strstr(char*s, char *sub)

  Ví dụ:

  ```
  char s1[50] = "mon hoc nhap mon lap trinh";
  char s2[50] = "nhap mon";
  cout<<strstr(s1, s2)<<endl;//xuất ra chuỗi "nhap mon lap trinh"
  ```

- Hàm **tolower**(): đổi một kí tự thành kí tự thường.

  int tolowr(int ch)

  Ví dụ:

  ```
  char ch='A';
  ch=tolower(ch);
  cout<<ch<<endl; //xuất ra ký tự 'a'
  ```

- Hàm **toupper**(): đổi tất cả các kí tự trong chuỗi thành hoa

  int tolowr(int ch)

  Ví dụ:

  ```
  char ch='a';
  ```

```
ch=toupper(ch);
cout<<ch<<endl; //xuất ra ký tự 'A'
```

III. NỘI DUNG THỰC HÀNH:

1/ Chương trình mẫu: (4 điểm)

Mô tả chương trình: Người dùng sẽ nhập một chuỗi từ bàn phím, chương trình thực hiện các thao tác sau:

- In ra chuỗi vừa nhập.
- In ra chuỗi đảo ngược của chuỗi vừa nhập.
- Thực hiện việc chèn một chuỗi sub vào chuỗi s tại vị trí pos và xuất ra kết quả.

➢ Cho đoạn mã trong file **lab8_thuvien.h** như sau:

Dòng	Mã lệnh	Ghi chú
1	#define _CRT_SECURE_NOWARNINGS	
2	#include <iostream>	
3	using namespace std;	
4	const int MAX=5;	
5		
6	void nhapChuoi(char *s);	
7	char* daoChuoi(char *s);	
8	char* chen(char *s, char *sub, int pos);	

➢ Cho đoạn mã trong file **lab8_caidat.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	#include "lab8_thuvien.h"	
2	void nhapChuoi(char *s)	
3	{	
4	cin.getline(s,MAX-1);	
5	}	
6	char* daoChuoi(char *s)	
7	{	
8	char *s2=new char[strlen(s)+1];	
9	int j=0;	
10	for (int i=strlen(s)-1; i>=0; i--)	
11	s2[j++]=s[i];	
12	s2[j] = '\0';	
13	return s2;	
14	}	
15	char* chen(char *s, char *sub, int pos)	

```
16  {
17      char *temp=new char[strlen(s)+strlen(sub)+1];
18      strcpy(temp,s);
19      char *s2=new char[MAX];
20      if(pos<0)
21          pos=0;
22      if (pos>strlen(s))
23          pos=strlen(s);
24      strcpy(s2,temp+pos);
25      temp[pos]='\0';
26      strcat(temp,sub);
27      strcat(temp,s2);
28      return temp;
29  }
```

➢ Cho đoạn mã trong file **lab8_chuongtrinh.cpp** như sau:

Dòng	Mã lệnh	Ghi chú

```
1   #include "lab8_thuvien.h"
2   int main()
3   {
4       char s[MAX];
5       char s2[MAX];
6       int viTri;
7       cout<<"Nhap chuoi: ";
8       nhapChuoi(s);
9       cout<<"Chuoi vua nhap la: "<<s<<endl;
10      cout<<"Chuoi dao nguoc la: "<<daoChuoi(s)<<endl;
11      cout<<"Nhap chuoi can chen: ";
12      nhapChuoi(s2);
13      cout<<"Nhap vi tri can chen: ";
14      cin>>viTri;
15      cout<<"Chuoi sau khi chen: "<<chen(s,s2,viTri)<<endl;
16      return 0;
17  }
```

Yêu cầu sinh viên: (2 điểm)

1. Biên dịch chương trình trên.
2. Cho biết kết quả in ra màn hình khi người dùng nhập dữ liệu sau: “dai hoc STU”
3. Nếu khai báo hàm nhapChuoi như sau được không? Giải thích.

void nhapChuoi(**char** s[]);

4. Mở file **lab8_caidat.cpp** và trả lời các câu hỏi sau:

a. Sửa lại dòng lệnh cin.getline(s,MAX-1); trong hàm **nhapChuoi** thành cin.getline(s,MAX); và chạy chương trình lần 1 với chuỗi 'abcd', chạy lần 2 với chuỗi 'abcde'. Hãy nhận xét kết quả 2 lần chạy.

b. Công dụng hàm, tham số, giá trị trả về của hàm **char*** daoChuoi(**char** *s)

c. Công dụng của lệnh **char** *s2=**new char**[strlen(s)+1]; trong hàm **daoChuoi**.

d. Vì sao phải dùng strlen(s)+1

e. Công dụng hàm, tham số, giá trị trả về của hàm **char*** chen(**char** *s, **char** *sub, **int** pos)

f. Công dụng của lệnh **if** (pos>strlen(s)) pos=strlen(s);

g. Công dụng của lệnh strcpy(s2,temp+pos); trong hàm **chen**.

h. Công dụng của lệnh temp[pos]='\0';

i. Công dụng của lệnh strcat(temp,sub);

5. Trong hàm daoChuoi, hãy thay vòng lặp while thành for
6. Trong hàm daoChuoi (file caidat_lab6), ta khai báo:

char s2[MAX] thay **char** *s2=**new char**[strlen(s)+1] được không?

7. Trong hàm chenChuoi, ta truyền tham số pos là số âm được không? Giải thích.

2/ Bài tập vận dụng: (3 điểm)

1. Bổ sung chương trình mẫu cho phép đổi chữ thành chữ HOA (không dùng hàm strupr)

   VD: Nhập "cong nghe thong tin" → "CONG NGHE THONG TIN"

2. Bổ sung chương trình mẫu cho phép in ra số các ký tự chữ HOA, số các ký tự chữ thường, số các ký tự chữ số.

   VD: Nhập "Cong Nghe Thong Tin 2020"

   → Kết quả:

   Số ký tự chữ HOA: 4
   Số ký tự chữ thường: 12
   Số ký tự chữ số: 4

3. Bổ sung chương trình mẫu cho phép đổi chữ HOA thành chữ thường, chữ thường thành chữ HOA, các ký tự khác giữ nguyên.

   VD: Nhập "Cong Nghe Thong Tin 2020"

   → Kết quả: "cONG nGHE tHONG tIN 2020"

3/ Bài tập tổng hợp: (3 điểm)

1. Bổ sung chương trình mẫu cho phép viết HOA đầu mỗi từ.

   VD: Nhập "cong nghe thong tin"

   → Kết quả: "Cong Nghe Thong Tin"
2. Tìm tất cả vị trí xuất hiện của chuỗi con Sub trong chuỗi mẹ S.

   VD: Nhập "abcdefabcdefabc"

   →Kết quả: Vị trí của "abc" là: 0 6 12
3. Thực hiện xóa num ký tự trong chuỗi s bắt đầu từ vị trí pos

   VD: Nhập "Cong Nghe Thong Tin"
   Pos = 5
   Num = 5
   →Kết quả: "Cong Thong Tin"

4/ Bài tập làm thêm: (2 điểm)

1. Trích một chuỗi con từ chuỗi mẹ S với vị trí trích là pos và trích num ký tự.

   VD: Nhập "Cong Nghe Thong Tin"

   Pos = 5

   Num = 4

   →Kết quả: "Nghe"
2. Bổ sung chương trình mẫu cho phép tìm kiếm và thay thế chuỗi con bằng chuỗi thay thế trong chuỗi mẹ.

   VD: Nhập "abcdefabcgth"

   Chuỗi tìm kiếm: "abc"

   Chuỗi thay thê: "A"

   →Chuỗi kết quả: "AdefAgth"
3. Bổ sung chương trình mẫu cho phép chuẩn hóa chuỗi. (xóa khoảng trắng đầu và cuối chuỗi, xóa những khoảng trắng dư thừa giữa các từ).

VD: Chuỗi mẹ: “ ab c de fg “

→Chuỗi đã chuẩn hóa: “ab c de fg”

4. Thực hiện việc tách một chuỗi họ tên thành họ lót và tên theo tên người Việt Nam. Ví du: họ tên = “Lê Triệu Ngọc Đức” ➔ họ lót = “Lê Triệu Ngọc” và tên = “Đức”
5. Thực hiện đảo ngược các từ có trong câu. Ví dụ: “cấm không được câu cá” → xuất ra: “cá câu được không cấm”
6. Thực hiện kiểm tra chuỗi s có tuần hoàn với chu kỳ n hay không (n là số nguyên dương). Ví dụ: s = “abcabcabcabc” là chuỗi tuần hoàn với chu kỳ n =3.

-HẾT LAB 8-

Bài 9: THAO TÁC TRÊN DANH SÁCH MẪU TIN

I. MỤC TIÊU:

Sau khi thực hành xong bài này, sinh viên có thể:

- Biết cách khai báo một mẫu tin (cấu trúc)
- Biết cách tạo mẫu tin có chứa các các thành phần khác nhau (kiểu số, kiểu chuỗi..)
- Biết cách tạo danh sách các mẫu tin (mỗi phần tử trong danh sách là một mẫu tin)

II. TÓM TẮT LÝ THUYẾT:

Nội dung	Ví dụ
1. Cách khai báo mẫu tin	

struct TenCauTruc
{
<Kieudulieu> tenthanhphan1;
<Kieudulieu> tenthanhphan2;
<Kieudulieu> tenthanhphan2;
};
	struct NhanVien
	{
	char maNhanVien[10];
	char hoTen[50];
	float luongCanBan;
	float heSoLuong;
	};

2. Cách khai báo danh sách mẫu tin	

struct TenCauTruc
{
<KieuDuLieu> tenThanhPhan1;
<KieuCauTruc> tenDanhSach[MAX];
};
	struct CongTy
	{
	int soNhanVien;
	NhanVien danhSach[MAX];
	};

3. Tìm lớn nhất (Luôn chọn được x_0 là phần tử đầu tiên thỏa điều kiện K)	Chọn x_0 thuộc S sao cho thỏa điều kiện K và gán Min = x_0 Duyệt x thuộc S (không chứa x_0) Nếu x thỏa K và Min > x thì Min = X Cuối duyệt
4. Tìm lớn nhất (Khó chọn được x_0 là phần tử đầu tiên thỏa điều kiện K (có thể có x_0 hoặc không có x_0)	Flag = false Duyệt x thuộc S Nếu x thỏa K Nếu not Flag thì Min = x; Flag = true Ngược lại nếu Min > x thì Min = x; Cuối nếu Cuối Duyệt
5. Kiểm tra tồn tại x thuộc tập S thỏa P(x): **(∃x ∈ S) (P(x))**	KT := false Duyệt x ∈ S Nếu P(x)thỏa thì

	KT := true Dừng quá trình duyệt S Cuối nếu Cuối duyệt

III. NỘI DUNG THỰC HÀNH:

1) Chương trình mẫu: (4đ)

Mô tả chương trình: Chương trình bao gồm các phần sau: Khai báo cấu trúc NhanVien, cấu trúc CongTy, các hàm nhập, xuất

➢ Cho đoạn mã trong file **lab9_thuvien.h** như sau:

Dòng	Mã lệnh	Ghi chú
1	#include <iostream>	
2	using namespace std;	
3	const int MAX = 50;	
4	struct NhanVien	
5	{	
6	char maNhanVien[10];	
7	char hoTen[50];	
8	float luongCanBan;	
9	float heSoLuong;	
10	};	
11	struct CongTy	
12	{	
13	int soNhanVien;	
14	NhanVien danhSach[MAX];	
15	};	
16	void nhapNhanVien(NhanVien &nhanVien);	
17	void xuatNhanVien(NhanVien nv);	
18	float tinhLuong(NhanVien nhanVien);	
19	void nhapCongTy(CongTy &cTy);	
20	void xuatCongTy(CongTy cTy);	
21	float tinhTongLuong(CongTy cTy);	
22	bool timNhanVienTheoMa(CongTy cTy, NhanVien &nhanVien);	
23	void hienThiMenu();	
24		

Bài 9: THAO TÁC TRÊN DANH SÁCH MẪU TIN

➢ Cho đoạn mã trong file **lab9_caidat.cpp** như sau:

Dòng	Mã lệnh	Ghi chú
1	#include "lab9_thuvien.h"	
2	// nhập thông tin 1 nhân viên	
3	void nhapNhanVien(NhanVien &nhanVien)	
4	{	
5	fflush(stdin);	
6	cout << "Ma nhan vien:";	
7	cin.getline(nhanVien.maNhanVien, 10);	
8	cout << "Ho ten:";	
9	cin.getline(nhanVien.hoTen, 50);	
10	cout << "Luong can ban:";	
11	cin >> nhanVien.luongCanBan;	
12	cout << "He so luong:";	
13	cin >> nhanVien.heSoLuong;	
14	}	
15	// xuất thông tin 1 nhân viên	
16	void xuatNhanVien(NhanVien nv)	
17	{	
18	cout << "Ma nhan vien:" << nv.maNhanVien << endl;	
19	cout << "Ho ten:" << nv.hoTen << endl;	
20	cout << "Luong can ban:" << nv.luongCanBan << endl;	
21	cout << "He so luong:" << nv.heSoLuong << endl;	
22	}	
23	// tính lương 1 nhân viên	
24	float tinhLuong(NhanVien nhanVien)	
25	{	
26	return nhanVien.luongCanBan*nhanVien.heSoLuong;	
27	}	
28	// nhập công ty (nhập danh sách nhân viên)	
29	void nhapCongTy(CongTy &cTy)	
30	{	
31	cout << "Nhap so luong nhan vien:";	
32	cin >> cTy.soNhanVien;	
33	cout << "Nhap danh sach nhan vien:" << endl;	
34	for (int i = 0; i < cTy.soNhanVien; i++)	
35	{	
36	cout << "Nhap nhan vien thu " << i << endl;	
37	nhapNhanVien(cTy.danhSach[i]);	
38	}	
39	}	
40	// xuất công ty (xuất danh sách nhân viên)	
41	void xuatCongTy(CongTy cTy)	

```
{
        cout << "\nSo luong nhan vien:" << cTy.soNhanVien <<
endl;
        cout << "Danh sach nhan vien:" << endl;
        for (int i = 0; i < cTy.soNhanVien; i++)
                xuatNhanVien(cTy.danhSach[i]);
}
// tính tổng lương của công ty
float tinhTongLuong(CongTy cTy)
{
        float tongLuong = 0;
        for (int i = 0; i < cTy.soNhanVien; i++)
                tongLuong = tongLuong +
tinhLuong(cTy.danhSach[i]);
        return tongLuong;
}
// hàm tìm nhân viên theo mã
bool timNhanVienTheoMa(CongTy cTy, NhanVien
&nhanVien)
{
        bool ketQua = false;
        char maCanTim[10];
        fflush(stdin);
        cout << "Nhap ma can tim:";
        cin.getline(maCanTim, 10);
        for (int i = 0; i < cTy.soNhanVien; i++)
        {
          if (_strcmpi(maCanTim,
         cTy.danhSach[i].maNhanVien) == 0){
                        ketQua = true;
                        nhanVien = cTy.danhSach[i];
                        break;
          }
        }
        return ketQua;
}
// hàm hienThiMenu
void hienThiMenu()
{
        int chon = 0;
        NhanVien nhanVien;
        CongTy cTy;
        CongTy cTyDaSapTangTheoHeSoLuong;
```

```
85          float tongLuong = 0;
86          float luongCaoNhat = 0;
87          CongTy dsNhanVienCoLuongCaoNhat;
88          bool ketQuaTimTheoMa = false;
89          bool ketQuaTimTheoTen = false;
90          CongTy dsNhanVienTimTheoTen;
91          do
92          {
93              cout << "\n0 - thoat chuong trinh:";
94              cout << "\n1 - nhap danh sach nhan vien:";
95              cout << "\n2 - xuat danh sach nhan vien:";
96              cout << "\n3 - tinh tong luong:";
97              cout << "\n4 - tim luong cao nhat:";
98              cout << "\n5 - tim nhan vien co luong cao nhat:";
99              cout << "\n6 - tim nhan vien theo ma:";
100             cout << "\n7 - tim nhan vien theo ten:";
101             cout << "\nVui long chon:";
102             cin >> chon;
103            switch (chon)
104             {
105             case 0:
106                 break;
107             case 1:
108                 nhapCongTy(cTy);
109                 break;
110             case 2:
111                 xuatCongTy(cTy);
112                 break;
113             case 3:
114                 tongLuong = tinhTongLuong(cTy);
115                 cout << "Tong luong cua cty la:" << tongLuong
116            << endl;
117                 break;
118             case 4:
119                 break;
120             case 5:
121                 break;
122             case 6:
123                 ketQuaTimTheoMa = timNhanVienTheoMa(cTy,
124 nhanVien);
125                 if (ketQuaTimTheoMa == true)
126                         xuatNhanVien(nhanVien);
127                 else
```

```
128                         cout << "Khong tim thay!";
129                     break;
130                 case 7:
131                     break;
132                 default:
133                     cout << "Ban chon sai, vui long chon lai!";
134                     break;
135             }
136         } while (chon != 0);
137 }
```

Yêu cầu: (2 điểm)

1. SV viết file **lab9_chuongtrinh.cpp** để thực thi chương trình trên.
2. Cho biết ý nghĩa của hàm _strcmpi tại dòng 69
3. Cho biết ý nghĩa của câu lệnh gán tại dòng 72
4. Cho biết ý nghĩa của câu lệnh break tại dòng 73

2) Bài tập vận dụng (3 điểm)

1. Viết hàm tìm lương cao nhất trong công ty.
2. Viết hàm tìm nhân viên có lương cao nhất đầu tiên
3. Viết hàm tìm nhân viên có lương cao nhất (tất cả nhân viên)

3) Bài tập tổng hợp: (3 điểm)

1. Viết hàm tìm nhân viên theo họ tên
2. Viết hàm tách tên nhân viên.
3. Viết hàm tìm nhân viên theo tên (theo tên chứ không phải theo họ)

4) Bài tập thêm: (2 điểm)

1. Viết hàm thêm 1 nhân viên mới
2. Viết hàm xoá nhân viên khi biết mã
3. Viết hàm cật nhận thông tin của nhân viên (trừ mã).

-HẾT LAB 9-

Bài 10: ÔN TẬP

ĐỀ THI THỬ

Bài 1: Cho cấu trúc dữ liệu dãy số như sau:

```
#define KTM 100
    struct DaySo {
    int list[KTM];
    int n;
};
```

a. Viết các hàm nhập/xuất dãy số trên. (2đ)

void nhapDS (DaySo &a) void xuatDS (DaySo a)

b. Viết hàm kiểm tra xem dãy số có phải là dãy giảm hay không (dãy giảm là dãy mà phần tử đứng sau luôn luôn NHỎ hơn phần tử đứng trước). Nếu đúng thì trả về true, ngược lại trả về false (2đ)
bool kiemTraGiam (DaySo a)

Bài 2: Cho cấu trúc dữ liệu ma trận vuông cấp n như sau:

```
struct MTVuong {
   int list[KTM][KTM];
   int n; //Cấp của ma trận
};
```

a. Viết hàm đếm các phần tử âm trên đường chéo phụ của ma trận vuông (2đ)

int demAmDCP (MTVuong a)

b. Viết hàm tìm phần tử nhỏ nhất trong tam giác dưới bên phải của ma trận vuông (1đ)

int timMinTamGiacDuoiPhai (MTVuong a)

Lưu ý: SV không cần viết các hàm nhập/xuất ma trận, SV có thể tự khởi gán giá trị các phần tử của ma trận.

Bài 3: Viết hảm đảo ngược chuỗi. Ví dụ: nhận chuỗi "abcde" trả về "edcba" (2đ)

char* daoChuoi (char* s)

Bài 4: Viết hàm **main** để gọi các yêu cầu ở trên. (1đ)

-HẾT-